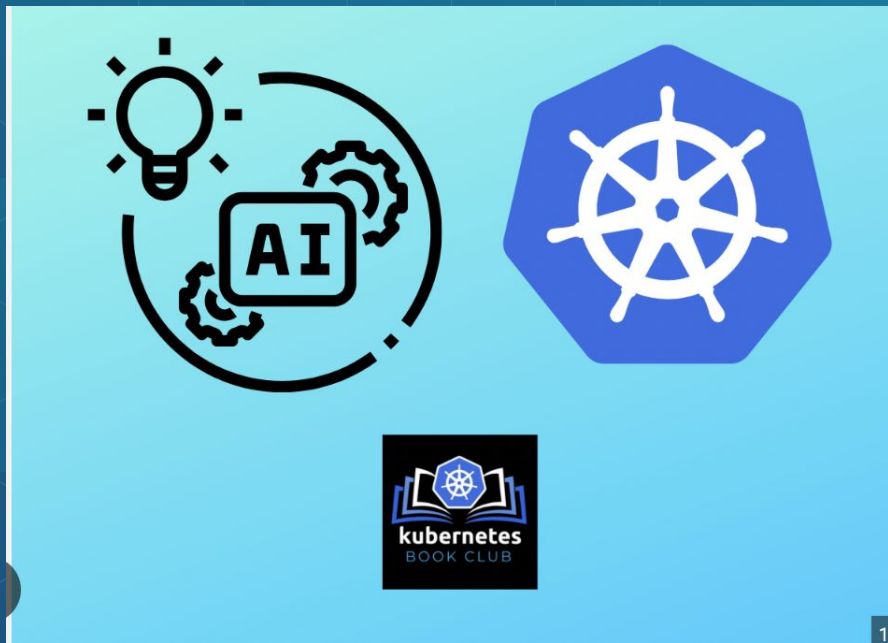


Machine learning on kubernetes

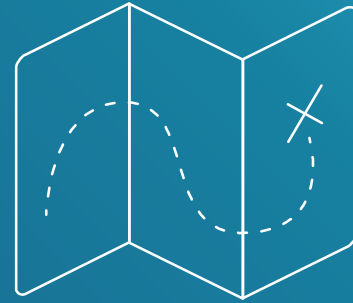


1.

Belsabel Woldemichael

Contents

- ◇ Introduction
- ◇ Design
- ◇ Implementation
- ◇ Test
- ◇ Enhancement Ideas
- ◇ Conclusion





Introduction

Objective: Deploy a Logistic Regression model using Kubernetes.

Key Steps:

- Study Logistic Regression.
- Set up a Kubernetes cluster on Google Cloud Platform.
- Deploy the model using Docker and Kubernetes.

Benefits:

- Scalable and reliable deployment.
- Simplified packaging with Docker.
- Efficient resource management with Kubernetes.



Design

Machine Learning Model:

- Logistic Regression for binary classification.
- Packaged with necessary dependencies in Docker.

Kubernetes Cluster:

- Hosted on Google Kubernetes Engine (GKE).
- Multi-node setup for high availability and scalability.

Deployment Workflow:

- **Build:** Create Docker image of the ML model.
- **Push:** Upload Docker image to Google Container Registry (GCR).
- **Deploy:** Use Kubernetes Deployment and Service YAML files to manage and expose the application.

Components:

- **Docker:** Containerised the ML application.
- **Kubernetes:** Orchestrates and scales containers.
- **Google Cloud Platform:** Provides infrastructure for running the Kubernetes cluster.



Implementation and Test

1. First, we create a Kubernetes cluster with three nodes if we deleted the existing ones from the previous HomeWorks. So, we enable GKE first and create Kubernetes cluster:

```
gcloud container clusters create kubia --num-nodes=1 --machine-type=e2-micro --region=us-west1
```

```
beilsabelwoldemichael@cs-312169370680-default:~$ gcloud services enable container.googleapis.com --project=cs570-big-data-429720
Operation "operations/acf.p2-942668407414-0b9a2a3b-b071-4766-8f00-b437e4235c86" finished successfully.
beilsabelwoldemichael@cs-312169370680-default:~$ gcloud container clusters create kubia --num-nodes=1 --machine-type=e2-micro --region=us-west1
Default change: VPC-native is the default mode during cluster creation for versions greater than 1.21.0-gke.1500. To create advanced routes based clusters, please pass the '--no-enable-ip-alias' flag
Note: The Kubelet readonly port (10255) is now deprecated. Please update your workloads to use the recommended alternatives. See https://cloud.google.com/kubernetes-engine/docs/how-to/disable-kubelet-readonly-port for ways to check usage and for migration instructions.
Note: Your Pod address range ('--cluster-ipv4-cidr') can accommodate at most 1008 node(s).
Creating cluster kubia in us-west1... Cluster is being health-checked (master is healthy)...done.
Created [https://container.googleapis.com/v1/projects/cs570-big-data-429720/zones/us-west1/clusters/kubia].
To inspect the contents of your cluster, go to: https://console.cloud.google.com/kubernetes/workload/_gcloud/us-west1/kubia?project=cs570-big-data-429720
kubeconfig entry generated for kubia.
NAME: kubia
LOCATION: us-west1
MASTER_VERSION: 1.29.6-gke.1038001
MASTER_IP: 35.230.4.105
MACHINE_TYPE: e2-micro
NODE_VERSION: 1.29.6-gke.1038001
NUM_NODES: 3
```

2. Check if the cluster and the nodes are correctly created and running. - Make sure your GKE cluster is running by:

`gcloud container clusters list`

```
See 'gcloud config set project [PROJECT_ID]' to change to a different project.  
belsabelwoldemichael@cs-312169370680-default:~$ gcloud container clusters list  
NAME: kubia  
LOCATION: us-west1  
MASTER_VERSION: 1.29.6-gke.1038001  
MASTER_IP: 35.230.4.105  
MACHINE_TYPE: e2-micro  
NODE_VERSION: 1.29.6-gke.1038001  
NUM_NODES: 3  
STATUS: RUNNING  
belsabelwoldemichael@cs-312169370680-default:~$
```

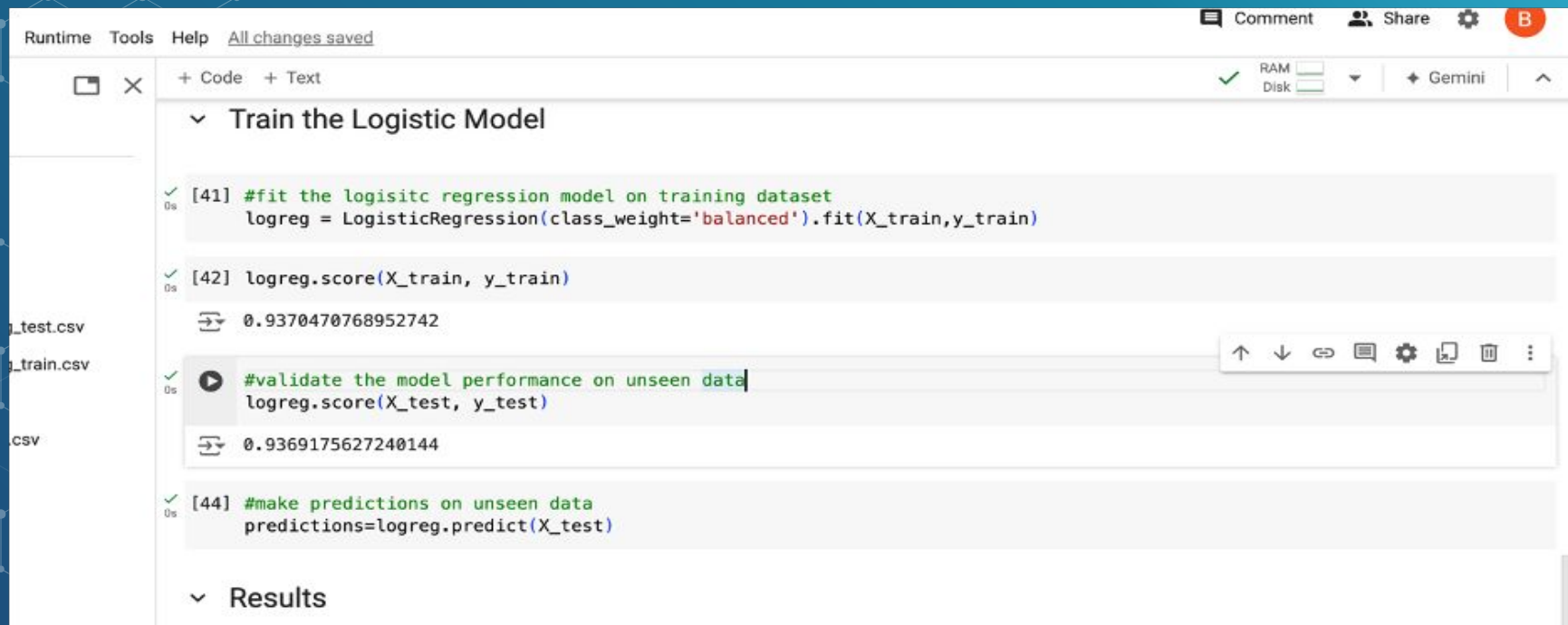
Or we can use minikube to run a single node using: minikube start

```
belsabelwoldemichael@cs-312169370680-default:~$ minikube start
* minikube v1.33.1 on Ubuntu 22.04 (amd64)
- MINIKUBE_FORCE_SYSTEMD=true
- MINIKUBE_HOME=/google/minikube
- MINIKUBE_WANTUPDATENOTIFICATION=false
* Automatically selected the docker driver. Other choices: none, ssh
* Using Docker driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.44 ...
* Downloading Kubernetes v1.30.0 preload ...
  > preloaded-images-k8s-v18-v1...: 342.90 MiB / 342.90 MiB 100.00% 102.87
  > gcr.io/k8s-minikube/kicbase...: 481.58 MiB / 481.58 MiB 100.00% 59.97 M
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.30.0 on Docker 26.1.1 ...
- kubelet.cgroups-per-qos=false
- kubelet.enforce-node-allocatable=""
- Generating certificates and keys ...
- Booting up control plane ...
- Configuring RBAC rules ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
- Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
belsabelwoldemichael@cs-312169370680-default:~$
```

Machine Learning Deployment Using Docker

Step 1. Train a machine learning model

- Build, train, and test a machine learning model.



The screenshot displays a Jupyter Notebook interface with a dark blue header bar. The header contains the text "Runtime Tools Help" and a link "All changes saved". On the right side of the header, there are icons for "Comment", "Share", and a red circle with a white "B". Below the header, there is a toolbar with a green checkmark, "RAM", "Disk", and a dropdown menu. The main content area is titled "Train the Logistic Model" and contains four code cells. The first cell (41) fits a Logistic Regression model on the training dataset. The second cell (42) calculates the score on the training data, resulting in 0.9370470768952742. The third cell (43) validates the model performance on unseen data, resulting in 0.9369175627240144. The fourth cell (44) makes predictions on the unseen data. The interface also shows a file explorer on the left with files named "g_test.csv", "g_train.csv", and "csv".

```
Runtime Tools Help All changes saved
```

+ Code + Text

✓ RAM
Disk

Comment Share B

Train the Logistic Model

```
[41] #fit the logisitc regression model on training dataset
logreg = LogisticRegression(class_weight='balanced').fit(X_train,y_train)
```

```
[42] logreg.score(X_train, y_train)
```

```
0.9370470768952742
```

```
[43] #validate the model performance on unseen data
logreg.score(X_test, y_test)
```

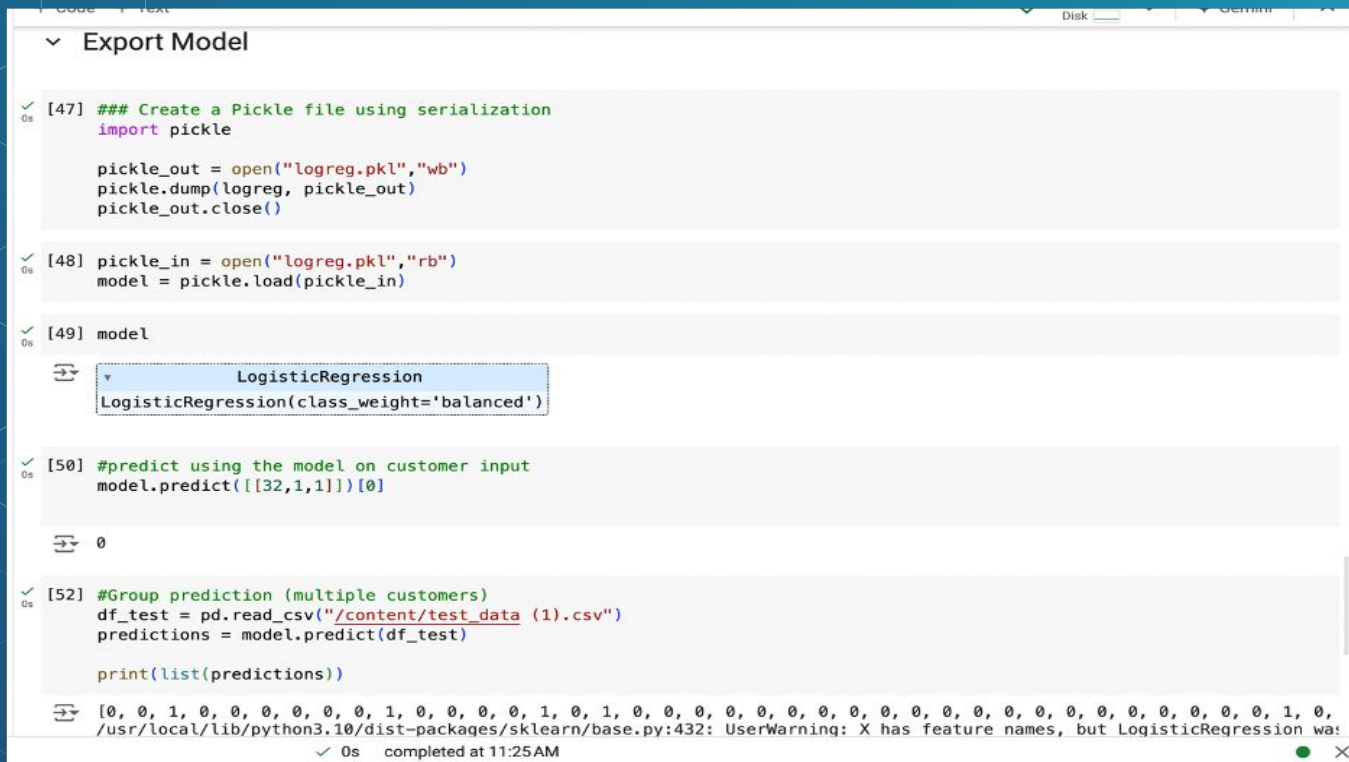
```
0.9369175627240144
```

```
[44] #make predictions on unseen data
predictions=logreg.predict(X_test)
```

Results

Step 2: Save and export the trained ML model.

- Once The model is trained, we need to save it using pickle/joblib in order to reuse it later during predictions.



```
[47] ### Create a Pickle file using serialization
import pickle

pickle_out = open("logreg.pkl", "wb")
pickle.dump(logreg, pickle_out)
pickle_out.close()

[48] pickle_in = open("logreg.pkl", "rb")
model = pickle.load(pickle_in)

[49] model
LogisticRegression(class_weight='balanced')

[50] #predict using the model on customer input
model.predict([[32,1,1]])[0]
0

[52] #Group prediction (multiple customers)
df_test = pd.read_csv("/content/test_data (1).csv")
predictions = model.predict(df_test)

print(list(predictions))
[0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0,
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:432: UserWarning: X has feature names, but LogisticRegression was
✓ 0s completed at 11:25AM
```


Step 3: Create a Flask app including the UI layer

```
GNU nano 6.2
# -*- coding: utf-8 -*-

from flask import Flask, request
import numpy as np
import pickle
import pandas as pd
from flasgger import Swagger

app = Flask(__name__)
Swagger(app)

# Load the pre-trained model
pickle_in = open("logreg.pkl", "rb")
model = pickle.load(pickle_in)

@app.route('/')
def home():
    return "Welcome to the Flask API!"

@app.route('/predict', methods=["GET"])
def predict_class():
    """
    Predict if Customer would buy the product or not.
    ---
    parameters:
      - name: age
        in: query
        type: number
        required: true
      - name: new_user
        in: query
        type: number
        required: true
      - name: total_pages_visited
        in: query
        type: number
        required: true
    responses:
      200:
        description: Prediction
    """
```

Step 4: Creating a custom Docker File for the app

1. Create Dockerfile using command - nano Dockerfile

```
GNU nano 6.2
FROM python:3.8-slim

# Set the working directory in the container
WORKDIR /app

# Copy the current directory contents into the container at /app
COPY . /app

# Expose port 5000 to the outside world
EXPOSE 5000

# Install any needed packages specified in requirements.txt
RUN pip install -r requirements.txt

# Define the command to run the application
CMD ["python", "flask_api.py"]
```

2. Create the requirements.txt file with the following contents. This file contains all the dependencies and libraries to be installed.

```
GNU nano 5.4
Flask==1.1.1
gunicorn==19.9.0
itsdangerous==1.1.0
Jinja2==2.10.1
MarkupSafe==1.1.1
Werkzeug==0.15.5
numpy==1.19.5 # Adjusted to a version before np.float deprecation
scipy>=0.15.1
scikit-learn==0.24.2 # Ensure compatibility with numpy version
matplotlib>=1.4.3
pandas>=0.19
flasgger==0.9.4
```


3. Then we upload all the files we created before to the terminal. Upload logreg.pkl, ML.ipynb, and flask_api.py files by clicking the three dots at the top-right of the Cloud Shell Terminal and then choose upload.

Upload

UPLOAD FILES OR FOLDERS FROM YOUR COMPUTER

☒ File ☐ Folder

Choose Files

logreg.pkl

×

ML.ipynb

×

online_sales.csv

×

test_data (1).csv

×

SELECT A DESTINATION DIRECTORY

Destination Directory

/home/belsabelwoldemichae

📁

Files can only be uploaded within the home directory. If the specified

Step 5. Run the app using a Docker container

1. We have to go to the terminal in the same directory where all the files are present and run the docker build command to build the Docker image from Dockerfile.
`sudo docker build -t ml_app_docker`

```
belsabelwoldemichael@cs-312169370680-default:~$ sudo docker build -t ml_app_docker .
[+] Building 73.7s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 423B
=> [internal] load metadata for docker.io/library/python:3.8-slim
=> [internal] load .dockerignore
=> => transferring context: 3B
=> [5/4] FROM docker.io/library/python:3.8-slim@sha256:48dba4e619b54ee99ef35695e2e2249392345161e0e7f5b21e7db6b6f0dd0df7
=> => resolve docker.io/library/python:3.8-slim@sha256:48dba4e619b54ee99ef35695e2e2249392345161e0e7f5b21e7db6b6f0dd0df7
=> => sha256:48dba4e619b54ee99ef35695e2e2249392345161e0e7f5b21e7db6b6f0dd0df7 10.41kB / 10.41kB
=> => sha256:b2d6266a63eaff44ce6a9149b80686ff6c17b0e4556d0320198ce85e9a241d 1.94kB / 1.94kB
=> => sha256:5208c64c1783c15b3d0ee357a3979688ae3faa21c6a13f4965e3530c66abbfcfc 6.93kB / 6.93kB
=> => sha256:efc2b5ad9e0c05hefa54239d53feca3569ccbef689aa5e5dhfc25da6c4df559 29.13MB / 29.13MB
=> => sha256:0d935f02ade5b557a3899b4161a3a7f7d88461fd62d550451b3884172a710962 3.51MB / 3.51MB
=> => sha256:fea4f2170757fa9e87b6421086bf5d32880bd00955941641106ea89519b5742 11.67MB / 11.67MB
=> => sha256:e5635d0cd4c514a4e76d97174785a452a1e81b87b6a8b38ce09c5ac2d135df8 230B / 230B
=> => extracting sha256:efc2b5ad9e0c05hefa54239d53feca3569ccbef689aa5e5dhfc25da6c4df559 2.9s
=> => sha256:eba330eb334fd723884e0b61fa4633bb792a3600f432a779301c8a7e4216d83 2.78MB / 2.78MB
=> => extracting sha256:0d935f02ade5b557a3899b4161a3a7f7d88461fd62d550451b3884172a710962 0.3s
=> => extracting sha256:fea4f2170757fa9e87b6421086bf5d32880bd00955941641106ea89519b5742 1.1s
=> => extracting sha256:e5635d0cd4c514a4e76d97174785a452a1e81b87b6a8b38ce09c5ac2d135df8 0.0s
=> => extracting sha256:eba330eb334fd723884e0b61fa4633bb792a3600f432a779301c8a7e4216d83 0.5s
=> [internal] load build context
=> => transferring context: 1.06GB
=> [2/4] WORKDIR /app
=> [3/4] COPY . /app
=> [4/4] RUN pip install -r requirements.txt
=> exporting image
=> exporting layers
=> => writing image sha256:6b16513470497161e30b45416d78951411bfdd16975ba5974b29b25450a51aebc
=> => naming to docker.io/library/ml_app_docker
belsabelwoldemichael@cs-312169370680-default:~$
```

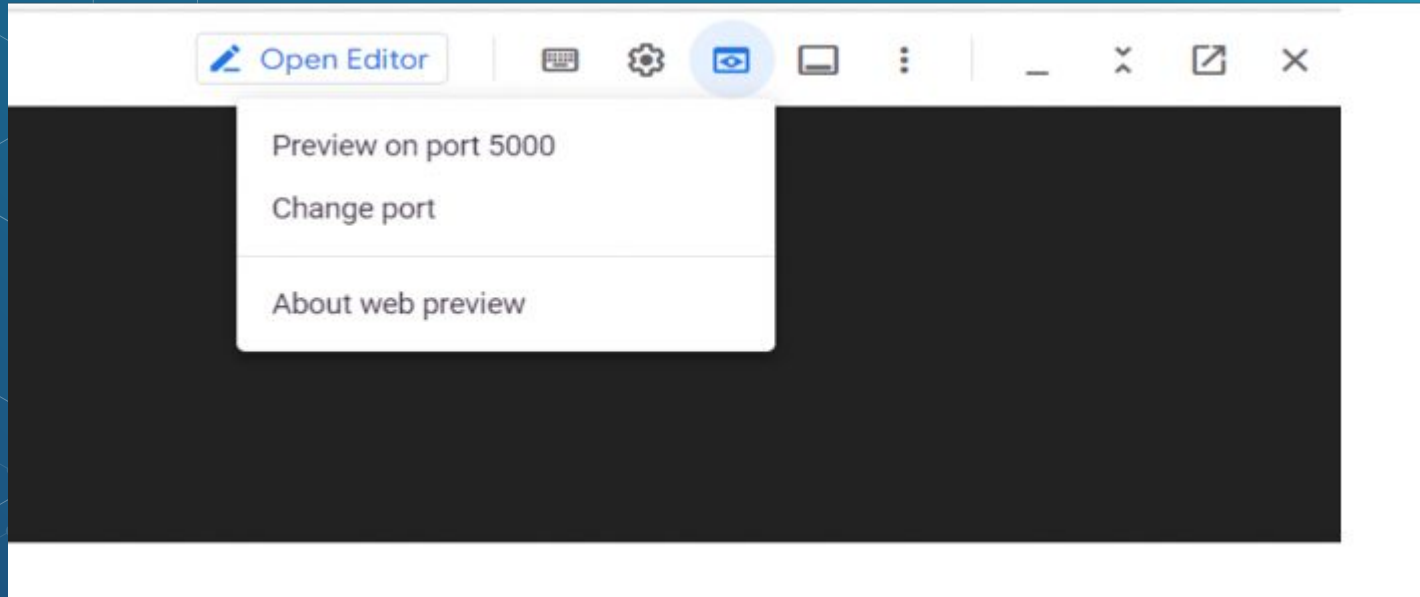
Transferred 1 item

2. Once all the commands in the Dockerfile get executed and we have the final image built from the Dockerfile, we can initiate the container to run our ML app. This command runs a Docker container from the ml_app_docker image:

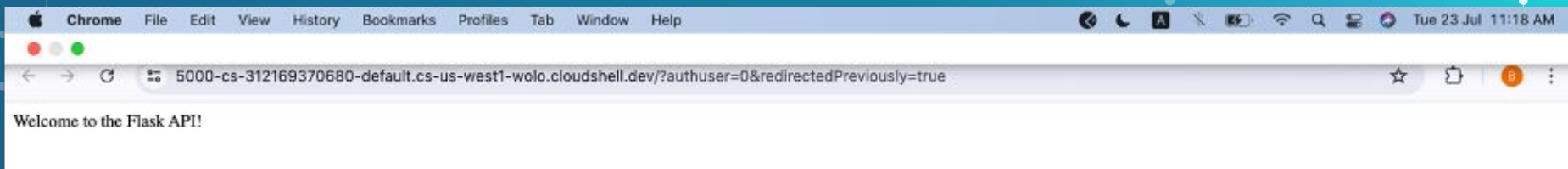
```
sudo docker container run -p 5000:5000 ml_app_docker
```

```
belsabelwoldemichael@cs-312169370680-default:~$ docker container run -p 5000:5000 ml_app_docker
* Serving Flask app "flask_api" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: on
/usr/local/lib/python3.8/site-packages/sklearn/base.py:310: UserWarning: Trying to unpickle estimator LogisticRegression from version 0.23.2 when using version 0.24.2. This might le
ad to breaking code or invalid results. Use at your own risk.
  warnings.warn(
* Running on http://0.0.0.0:5000/ (Press CTRL+C to quit)
* Restarting with stat
/usr/local/lib/python3.8/site-packages/sklearn/base.py:310: UserWarning: Trying to unpickle estimator LogisticRegression from version 0.23.2 when using version 0.24.2. This might le
ad to breaking code or invalid results. Use at your own risk.
  warnings.warn(
* Debugger is active!
* Debugger PIN: 298-690-507
172.17.0.1 - - [23/Jul/2024 18:17:51] "GET /?authuser=0sredirectedPreviously=true HTTP/1.1" 200 -
172.17.0.1 - - [23/Jul/2024 18:17:51] "GET /favicon.ico HTTP/1.1" 404 -
□
```

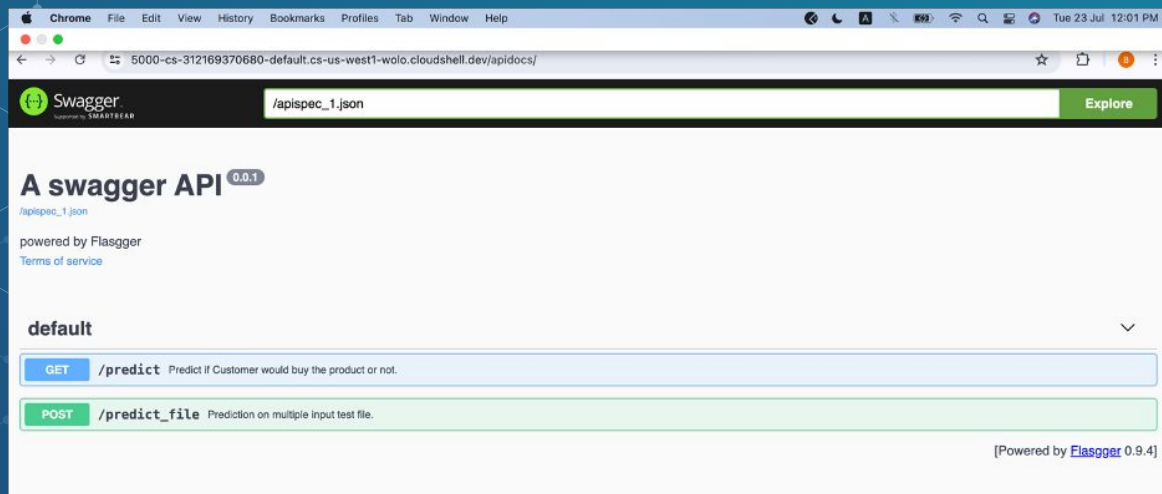
3. In the right-upper side of the terminal, click the web preview option from the menu and then click Preview on port 5000. Change port if it is not 5000 by default.



4. After clicking on Change and Preview you will see this message in the web preview.



5. Add /apidocs/ at the end of the link to access to load the Swagger UI page.



6. Click GET and then click Try it out in the top-right corner of the GET box/

7. Fill values for the input parameters and then click Execute.

default ▼

GET /predict Predict if Customer would buy the product or not.

Parameters Cancel

Name	Description
age • required number (query)	23
new_user • required number (query)	06
total_pages_visited • required number (query)	7

Click to go back, hold to see history

8. Upon the execution call, the request goes to the app, and predictions are made by the model. The result of the model prediction is displayed in the Prediction section of the page as follows.

Responses Response content type

Curl

```
curl -X GET "https://5000-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev/predict?age=23&new_user=066total_pages_visited=7" -H "accept: application/json"
```

Request URL

```
https://5000-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev/predict?age=23&new_user=066total_pages_visited=7
```

Server response

Code	Details
200	Response body <pre>Model prediction is [0]</pre> Download Response headers <pre>content-length: 23 content-security-policy: frame-ancestors 'self' https://80-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev https://cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev https://ide.cloud.google.com https://shell.cloud.google.com https://ssh.cloud.google.com https://console.cloud.google.com content-type: text/html; charset=utf-8 date: Tue, 23 Jul 2024 19:03:07 GMT server: Werkzeug/0.15.5 Python/3.8.19</pre>

Responses

9. The next prediction that can be done is for a group of customers (test data) via a post request.

The screenshot shows a REST client interface with the following sections:

- POST /predict_file** Prediction on multiple input test file.
- Parameters** section with a **Cancel** button.
- file** (required) parameter with a **file** type and *(formData)* label. It includes a **Choose File** button and the text "No file chosen".
- Execute** button.
- Responses** section with a **Response content type** dropdown set to **application/json**.
- Code** and **Description** table:

Code	Description
200	Test file Prediction

10. Upload the test data file containing the same parameters in a similar order. The model would make the prediction, and the results would be displayed upon execute as following.

file * required

file

(formData)

Choose File

test_data.csv

Execute

Clear

Responses

Response content type

application/json

Curl

```
curl -X POST "https://5000-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev/predict_file" -H "accept: application/json" -H "Content-Type: multipart/form-data" -F "file=@test_data.csv;type=text/csv"
```

Request URL

```
https://5000-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev/predict_file
```

Server response

Code

Details

200

Response body

```
[0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Download

Response headers

```
access-control-allow-credentials: true
access-control-allow-methods: GET,POST,OPTIONS,PATCH,DELETE
access-control-allow-origin: https://5000-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev
content-length: 150
content-security-policy: frame-ancestors 'self' https://80-cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev https://cs-312169370680-default.cs-us-west1-wolo.cloudshell.dev https://ide.cloud.google.com https://shell.cloud.google.com https://ssh.cloud.google.com https://console.cloud.google.com
content-type: text/html; charset=utf-8
date: Tue, 23 Jul 2024 19:04:07 GMT
server: Werkzeug/0.15.5 Python/3.8.19
```

Step 6: Stopping/killing the running container

1. Use docker ps to list running Docker containers

```
belsabelwoldemichael@cs-312169370680-default:~$ ls
convert_data.sh      kubia-ingress-multiple.yaml  kubia-svc-headless.yaml  movies.csv             recommendation.py        'test_data (1).csv'
Dockerfile           kubia-ingress-tls.yaml      kubia-svc-nodeport.yaml  online_sales.csv       recommendation.pyNY     tls.cert
Edited_data.csv      kubia-ingress.yaml          kubia-svc.yaml           ratings.csv             requirements.txt         tls.key
'flask_api_(1).py'    kubia-rc-readiness.yaml     logreg.pkl               README-cloudshell.txt  spark-3.5.1-bin-hadoop3.tgz  u.data
flask_api.py         kubia-rc.yaml               ML.ipynb                 recommendation_engine_movielens.py  test.data

belsabelwoldemichael@cs-312169370680-default:~$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	NAMES	CREATED	STATUS	PORTS
d05e90b60f9f	flask-api	"python flask_api.py"	fervernt gagarin	15 minutes ago	Up 15 minutes	0.0.0.0:5000->5000/tcp
6919692644b6	gcr.io/k8s-minikube/kicbase:v0.0.44	"/usr/local/bin/entr..."	minikube	About an hour ago	Up About an hour	127.0.0.1:32768->22/tcp, 127.0.0.1:32769->2376/tcp, 127.0.0.1:32770->5000/tcp, 127.0.0.1:32771->8443/tcp, 127.0.0.1:32772->32443/tcp

2. Use the command - docker kill to kill the running container as follows.

```
belsabelwoldemichael@cs-312169370680-default:~$ docker kill d05e90b60f9f
d05e90b60f9f
belsabelwoldemichael@cs-312169370680-default:~$ █
```



Enhancement Ideas

- **Additional Models:** Deploy algorithms like Decision Trees and XGBoost. Expand beyond Logistic Regression for more versatility.
- **CI/CD Integration:** Automate builds and deployments using Jenkins or GitHub Actions. Streamline updates and reduce manual errors.
- **Monitoring and Logging:** Integrate Prometheus/Grafana for performance metrics. Use ELK Stack for centralized logging and easier debugging.
- **Security:** Implement RBAC for access control and TLS/SSL for secure communication. Protect your deployment from unauthorized access.



Conclusion

- End-to-End Deployment: Successfully deployed a Logistic Regression model using Kubernetes and Docker.
- Scalable Infrastructure: Utilized Google Kubernetes Engine for scalable and reliable model management.
- Enhanced Efficiency: Integrated CI/CD pipelines, monitoring, and security measures to streamline operations.
- Robust Solution: Provided a scalable, efficient framework for managing machine learning models in production.

References

A Guide to Deploying Machine Learning Models on Kubernetes

What is Kubernetes for MLOps

Github Link

https://github.com/BelsabelTekle/Cloud_Computing/tree/main/Kubernetes/Machine_learning